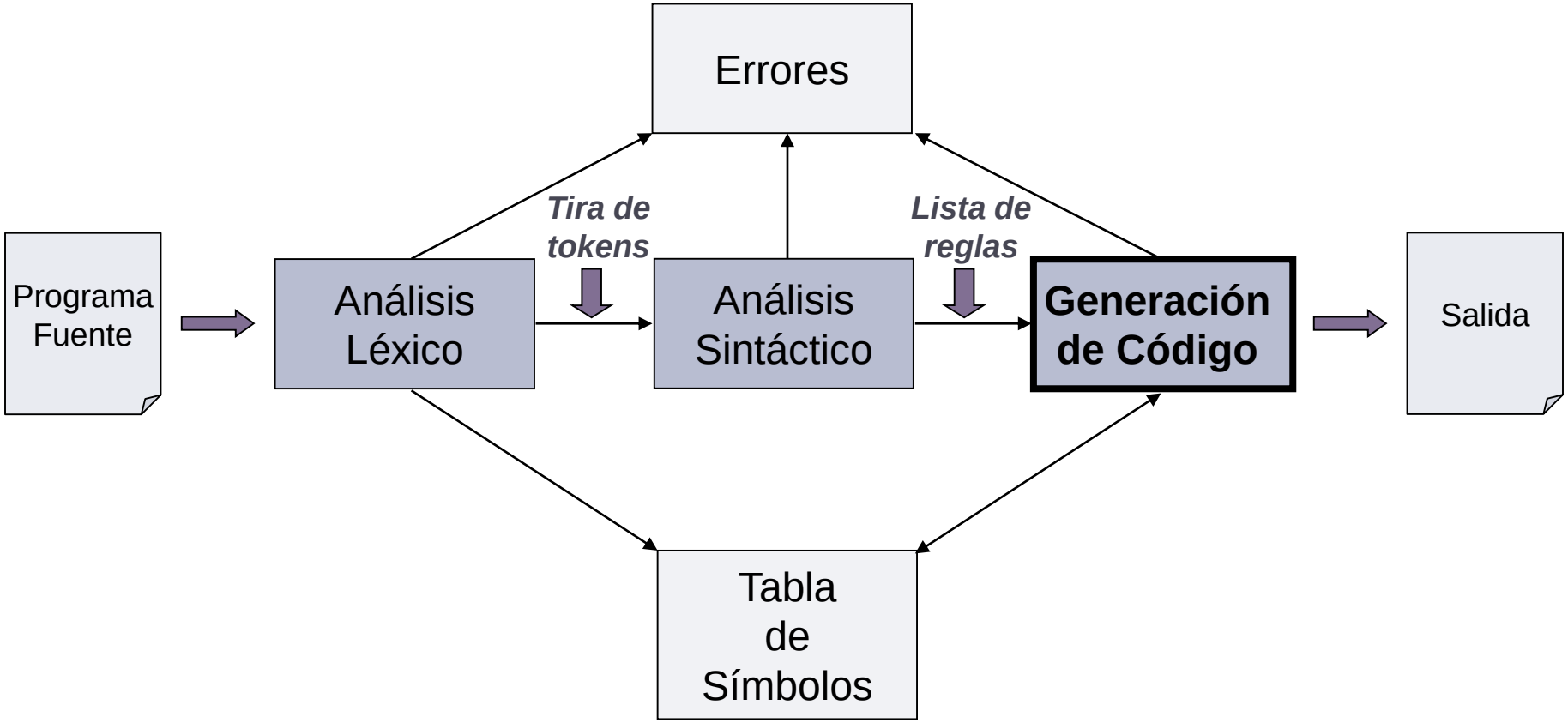
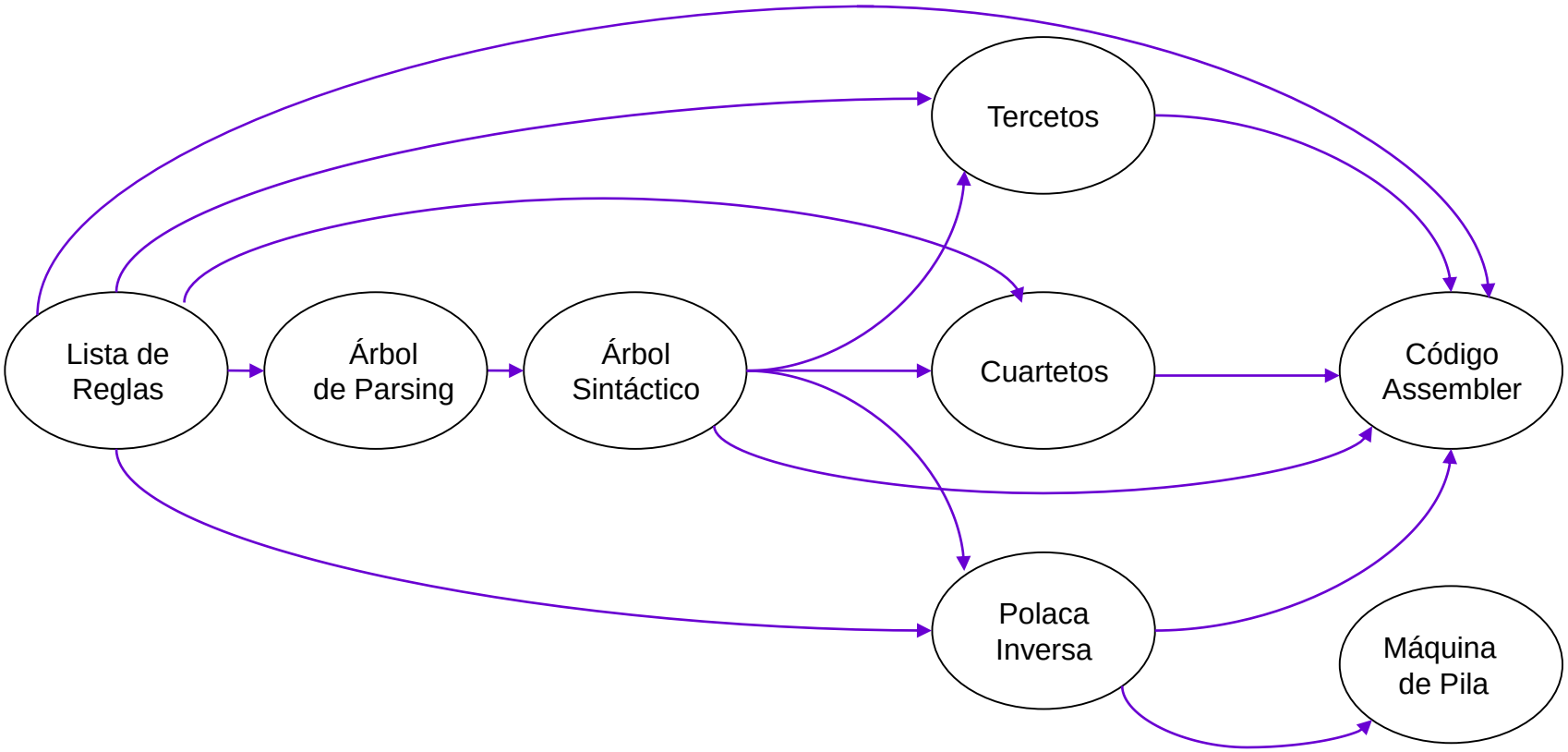
Diseño de Compiladores I
Compiladores e Intérpretes

Caminos para la Generación de Código

Fases de la Compilación



Generación de Código



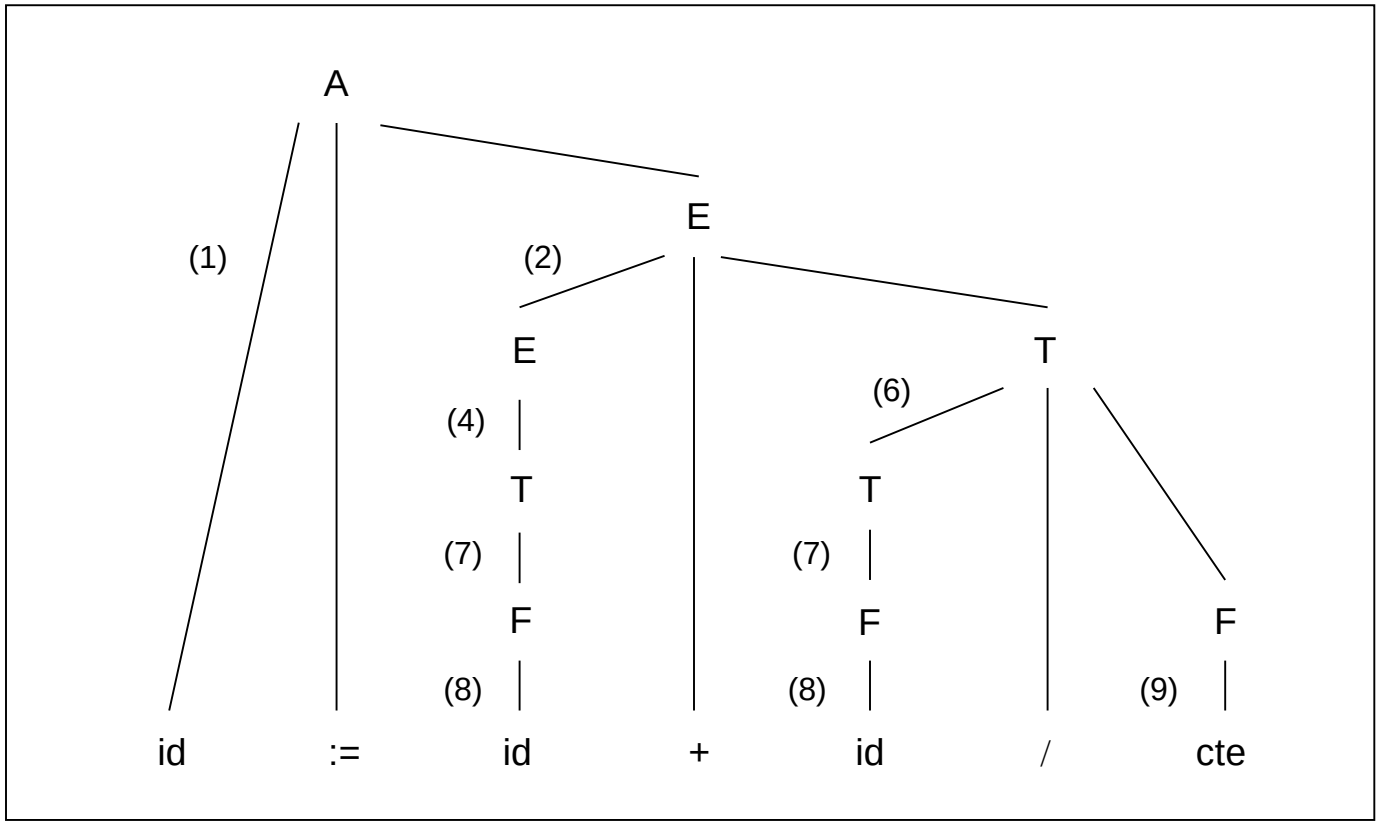
Representaciones:
Gramática

1) $A \rightarrow id := E$
2) $E \rightarrow E + T$
3) $E \rightarrow E - T$
4) $E \rightarrow T$
5) $T \rightarrow T * F$
6) $T \rightarrow T / F$
7) $T \rightarrow F$
8) $F \rightarrow id$
9) $F \rightarrow cte$



Representaciones:
Árbol de Parsing

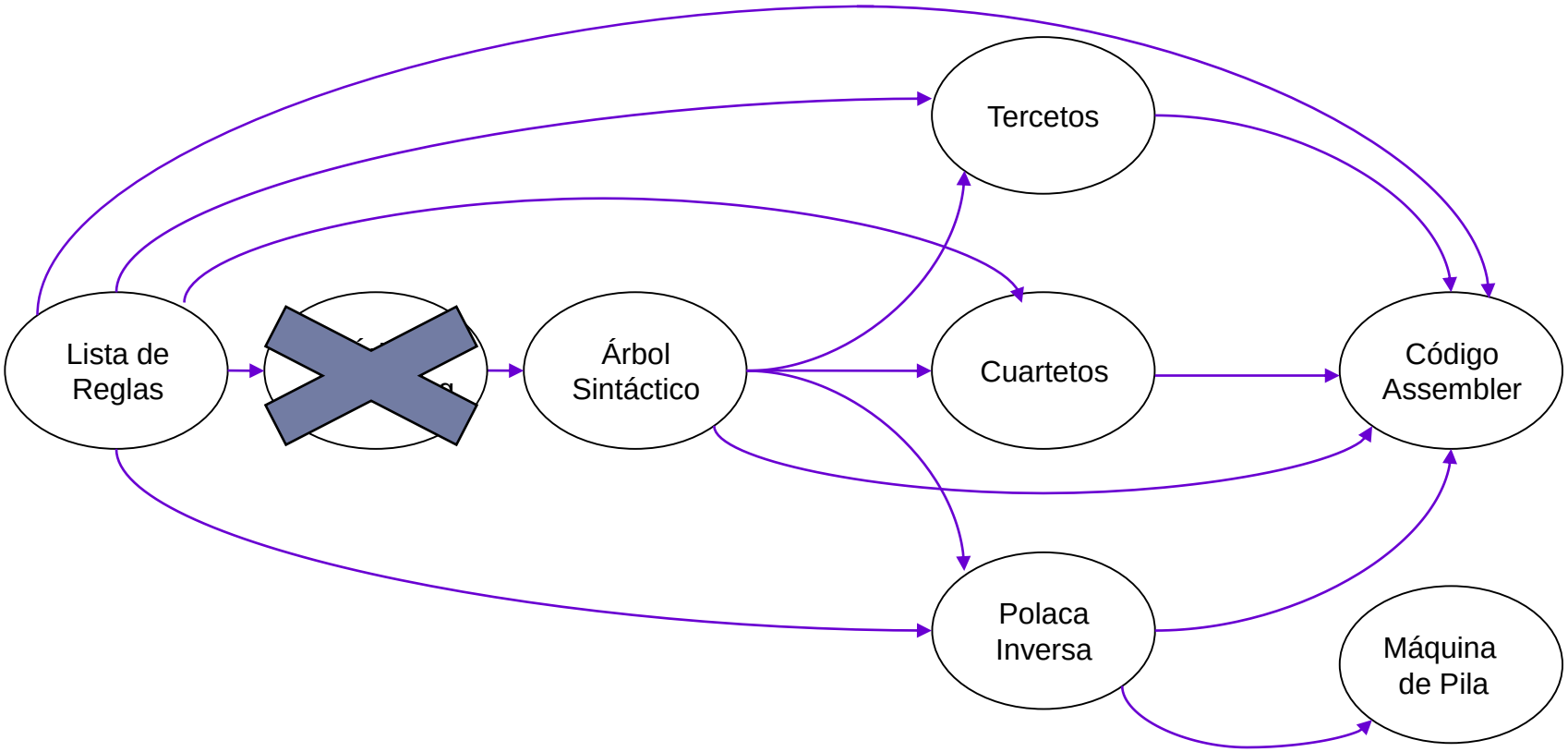
Tasa := Base + Recargo / 100.0 → id := id + id / cte



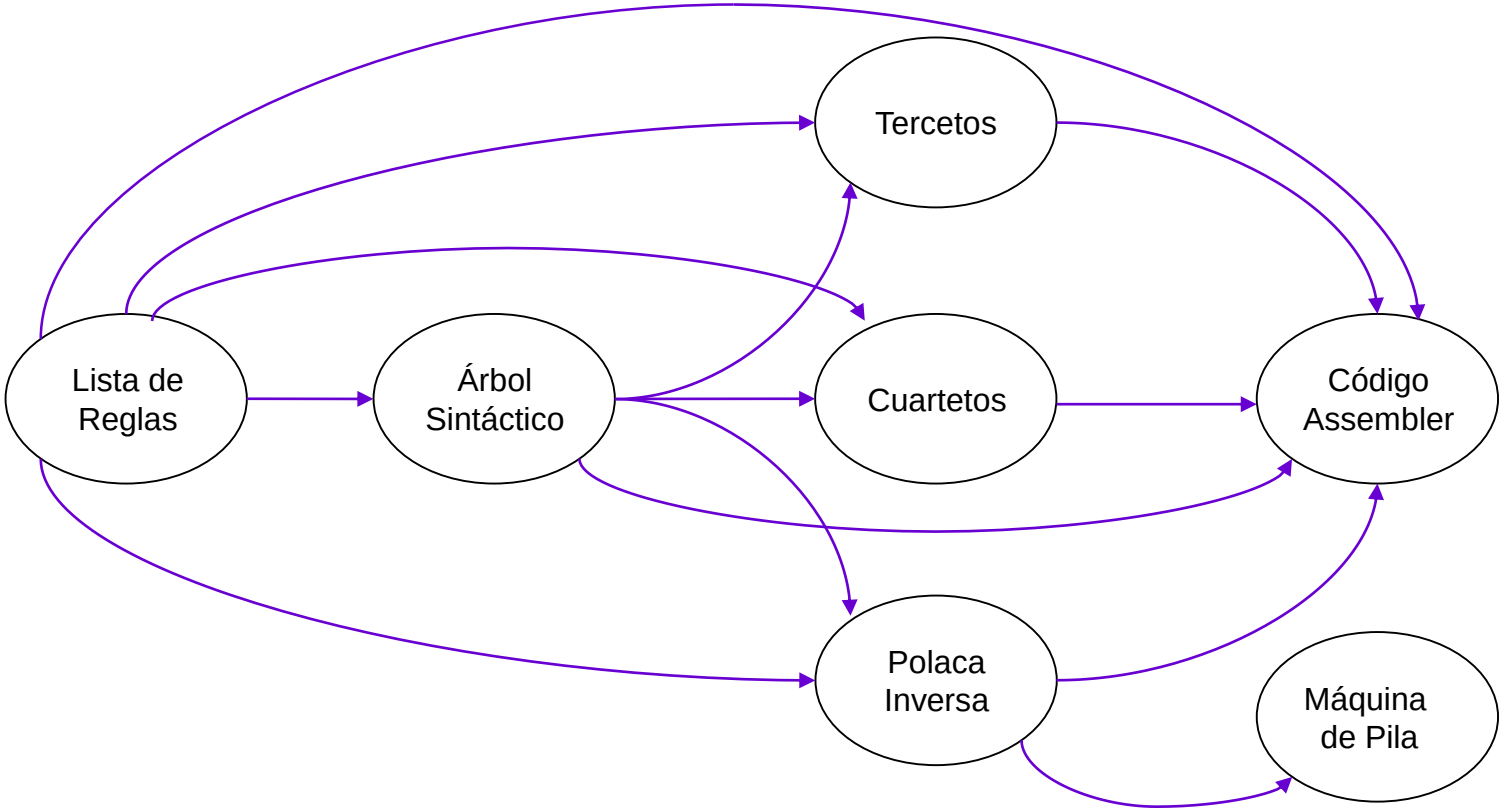
1) A → id := E
2) E → E + T
3) E → E – T
4) E → T
5) T → T * F
6) T → T / F
7) T → F
8) F → id
9) F → cte

Lista de reglas: 8 7 4 8 7 9 6 2 1

Generación de Código



Generación de Código



Representaciones: Árbol Sintáctico

Es una representación comprimida del árbol de parsing.

Tasa := Base + Recargo / 100.0 → ID := ID + ID / CTE

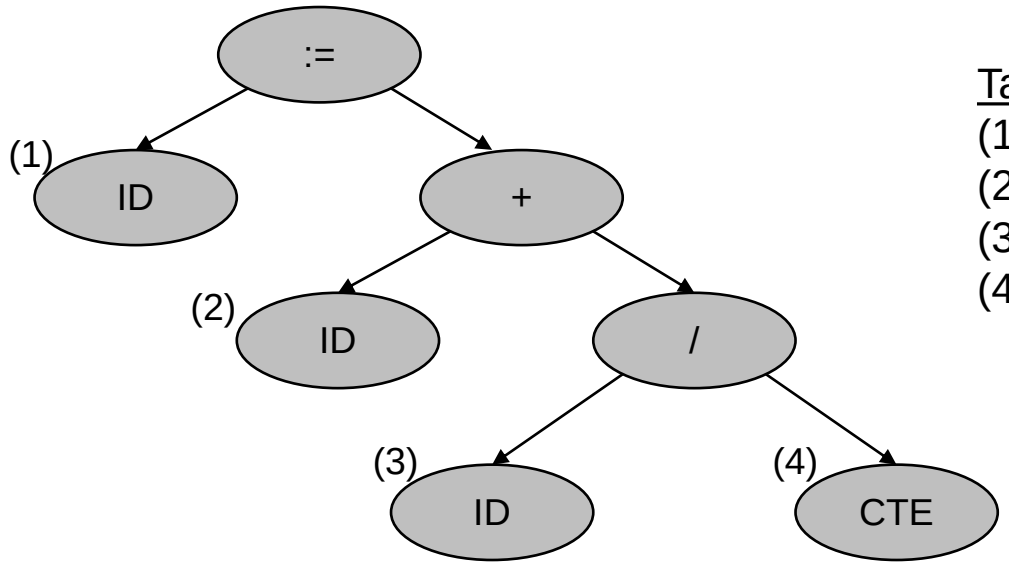


Tabla de Símbolos

(1) Tasa	real
(2) Base	real
(3) Recargo	real
(4) 100.0	real

Representaciones: Tercetos

Es una representación constituida por códigos de 3 direcciones

(operador, operando 1, operando 2)

Tasa := Base + Recargo / 100.0 → ID := ID + ID / CTE

14. . . .
15. (/, Recargo, 100.0)
16. (+, Base, [15])
17. (:=, Tasa, [16])
18. . . .

Tabla de Símbolos

(1) Tasa	real
(2) Base	real
(3) Recargo	real
(4) 100.0	real

Los operandos son referencias a la Tabla de Símbolos, o a otros tercetos.



Representaciones: Cuartetos

Es una representación constituida por códigos de 4 direcciones

(operador, operando 1, operando 2, resultado)

Tasa := Base + Recargo / 100.0 → ID := ID + ID / CTE

14. …
15. (/, Recargo, 100.0, aux1)
16. (+, Base, aux1, aux2)
17. (:=, Tasa, aux2, -)
18. …

Tabla de Símbolos

(1) Tasa	real
(2) Base	real
(3) Recargo	real
(4) 100.0	real

Los operandos son referencias a la Tabla de Símbolos

Representaciones:
Polaca Inversa (Notación Postfija)

Para esta representación:

Operadores binarios: op1 operador op2 → op1 op2 operador

Operadores unarios: operador op1 → op1 operador

Tasa := Base + Recargo / 100.0 → ID := ID + ID / CTE

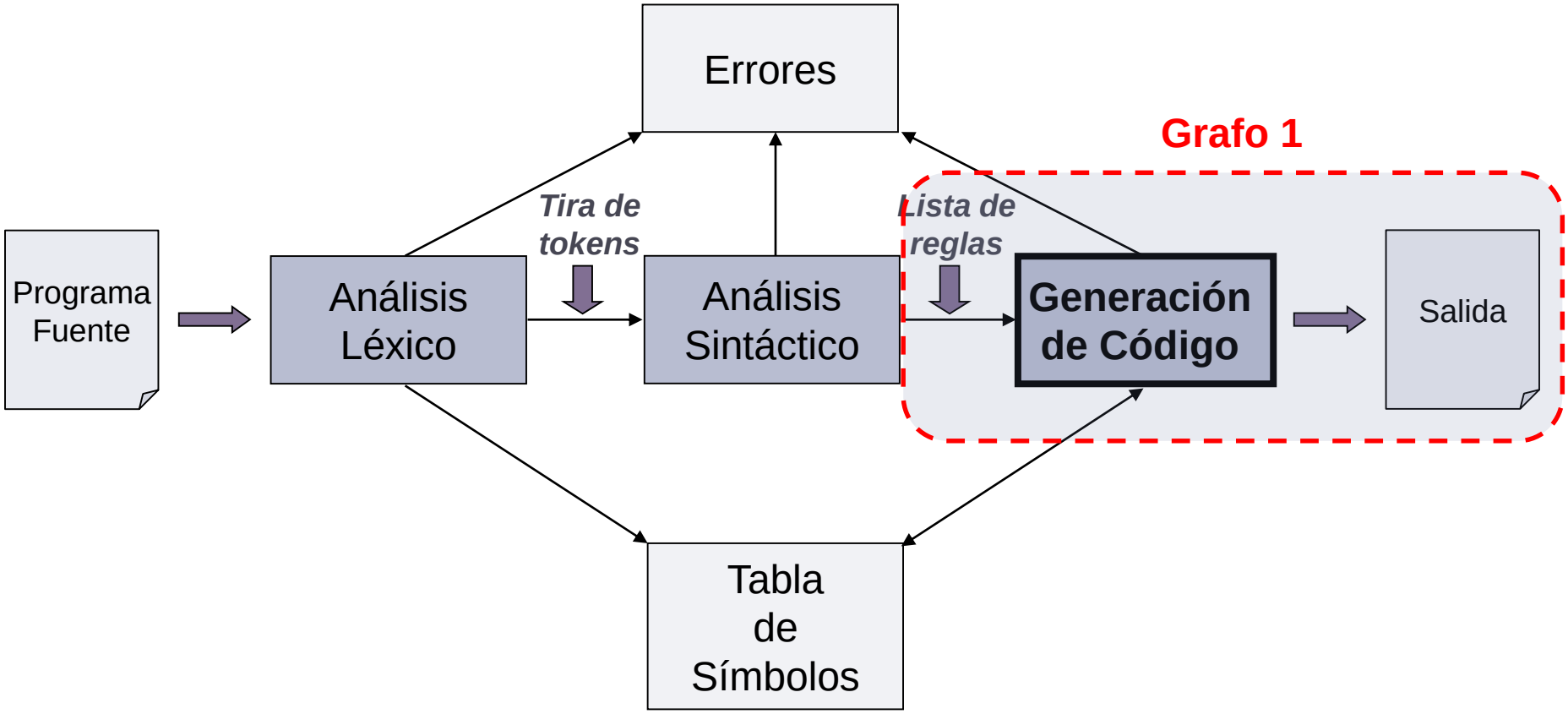
...	Base	Recargo	100.0	/	+	Tasa	:=	...
21	22	23	24	25	26	27	28	29

Los operandos son referencias a la Tabla de Símbolos

Tabla de Símbolos

(1) Tasa	real
(2) Base	real
(3) Recargo	real
(4) 100.0	real

Fases de la Compilación

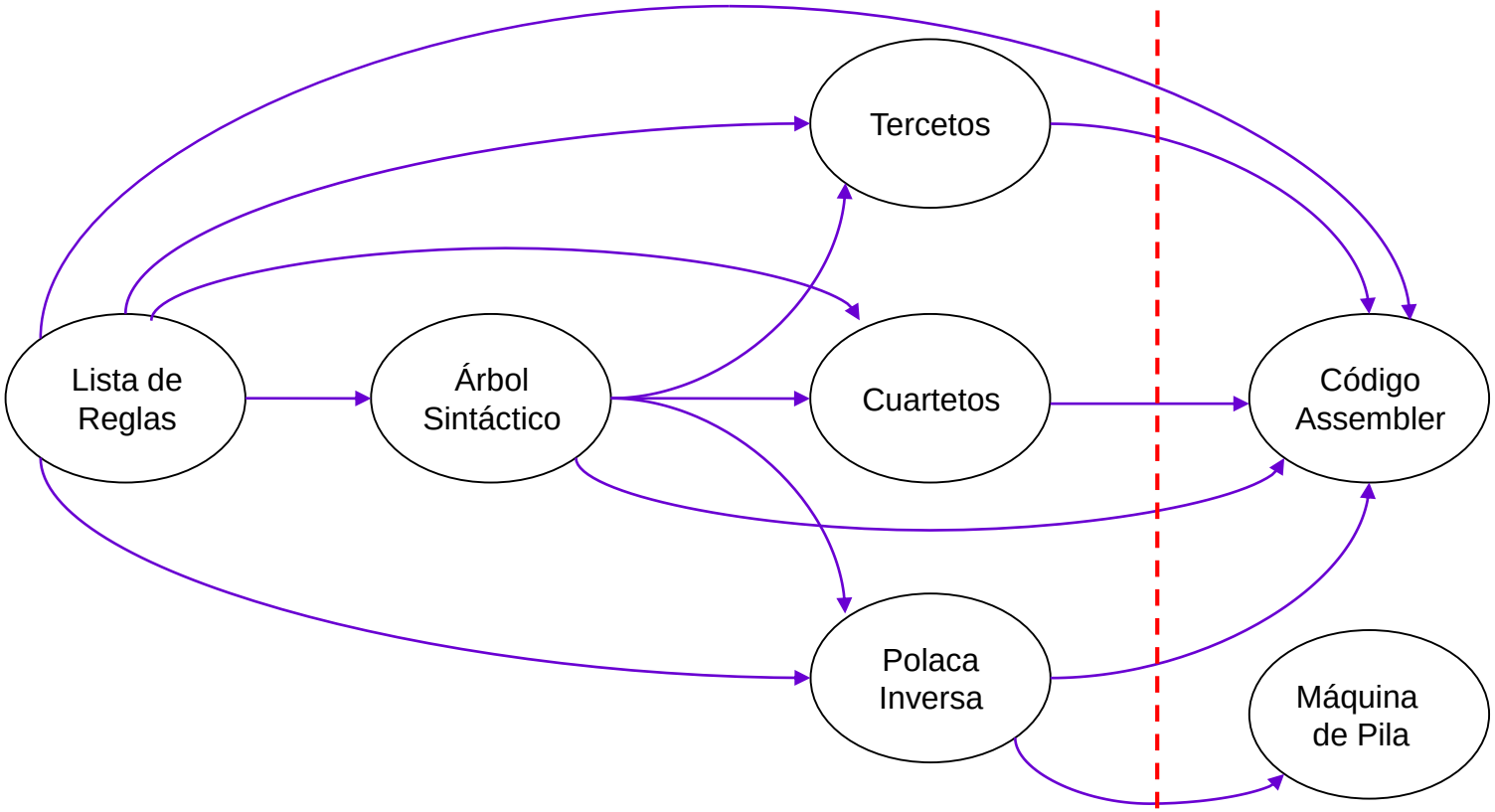


Generación de Código
Grafo 1

Sentencias ejecutables
x := x + 2;

Sólo depende del lenguaje

Depende del S.O. y de la máquina



Representaciones: Código Assembler

Esta representación depende del Sistema Operativo, y la máquina para la cual se genera.

Tasa := Base + Recargo / 100.0 → ID := ID + ID / CTE

```
…
FLD Base
FLD Recargo
FDIV 100.0
FADD
FST Tasa
…
```

(Ejemplo de código Assembler utilizando el coprocesador matemático de Pentium)

Máquina de Pila

- No se traduce → Se interpreta
- Técnica en desuso.



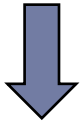
- Se retoma:
  - Smalltalk (se interpreta mucho)
  - Java (se compila más y luego se interpreta)

Comparación de caminos

Compilador rápido (el más rápido)

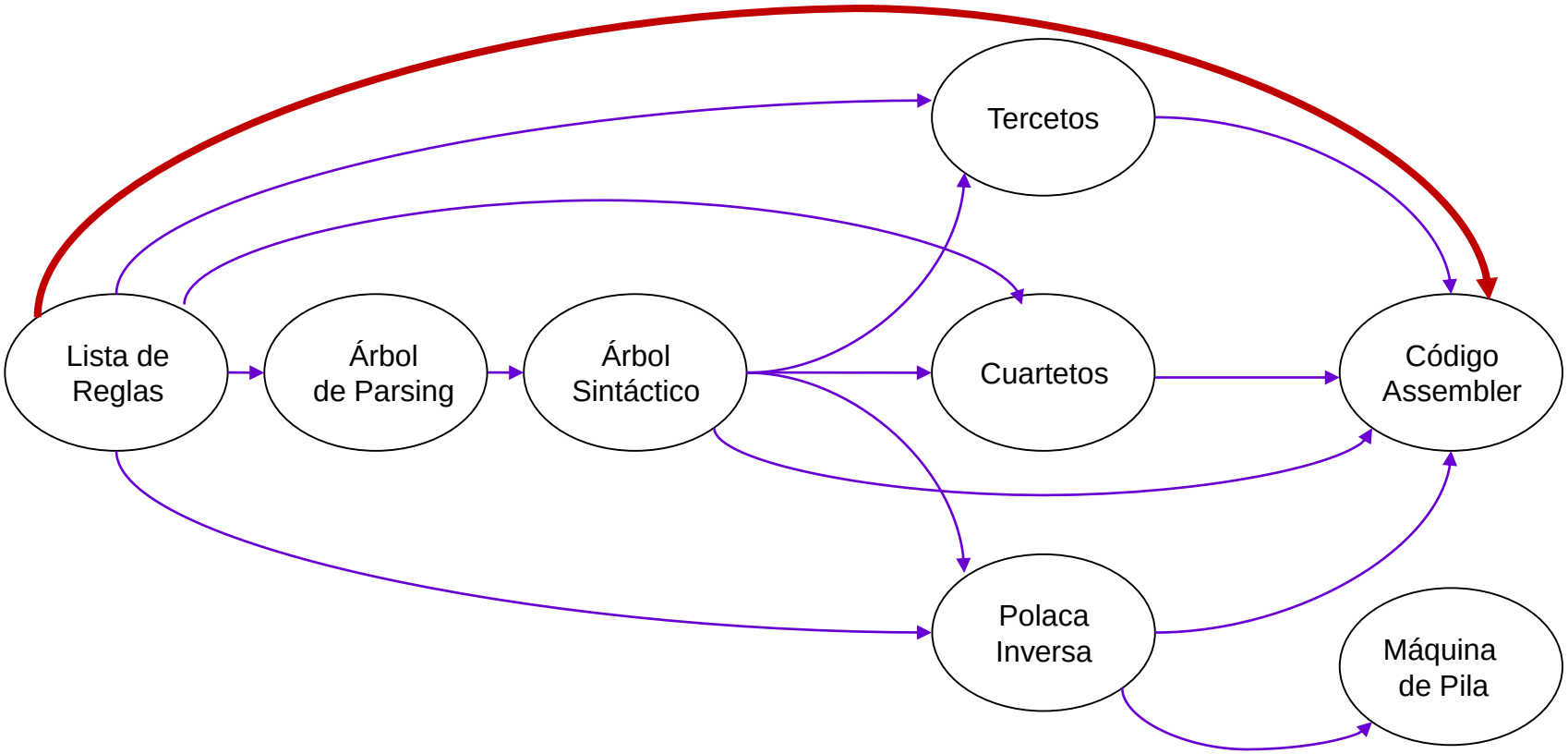


Salida ejecutable grande y lenta



Lista de Reglas → Assembler

Generación de Código

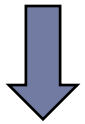


Comparación de caminos

Salida ejecutable rápida



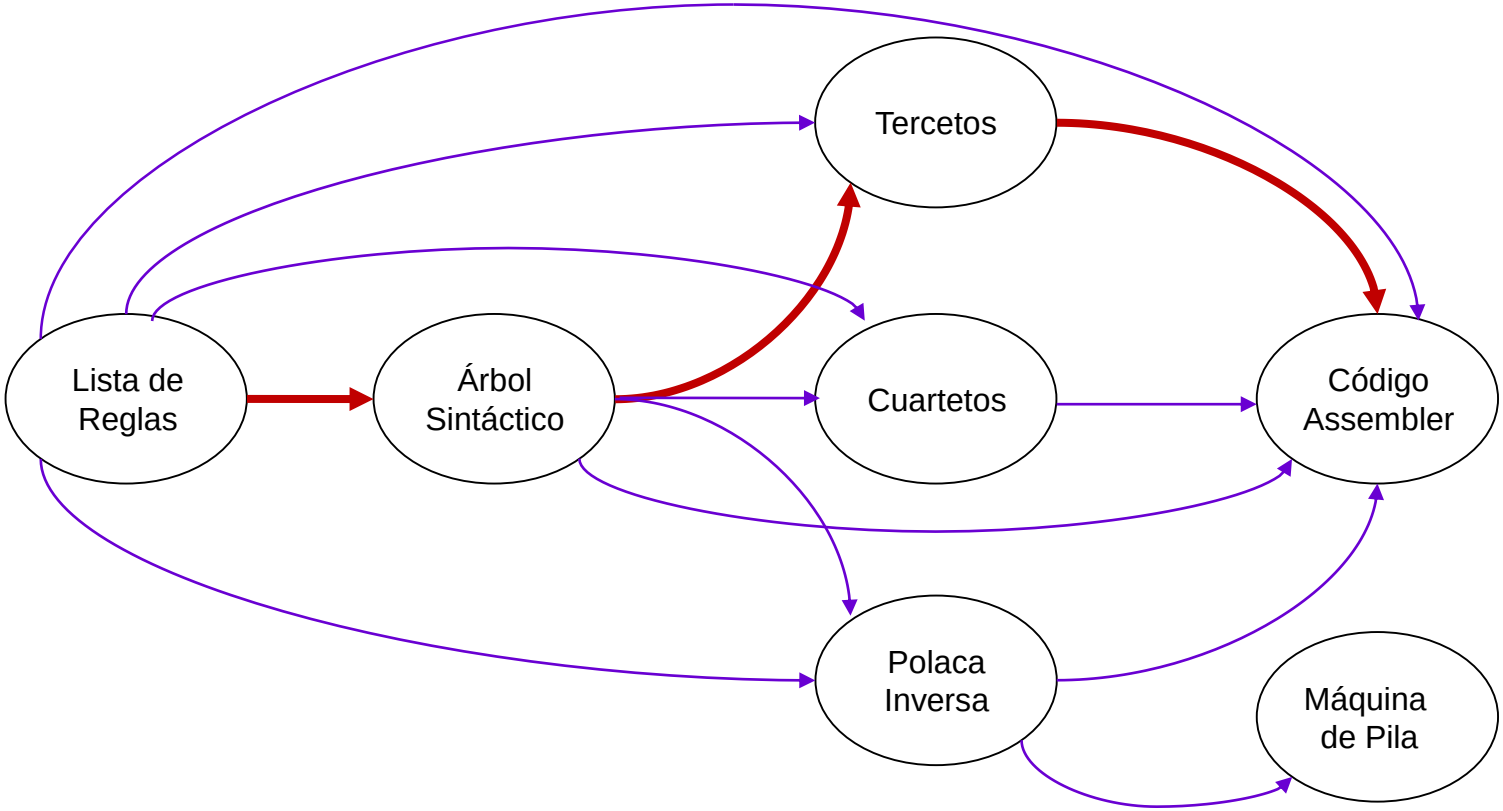
Compilador lento



L.Reglas → Á.Sintáctico → Tercetos → Assembler

(Optimizaciones)

Generación de Código



Comparación de caminos

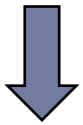
Desarrollo Rápido



Compilador relativamente rápido

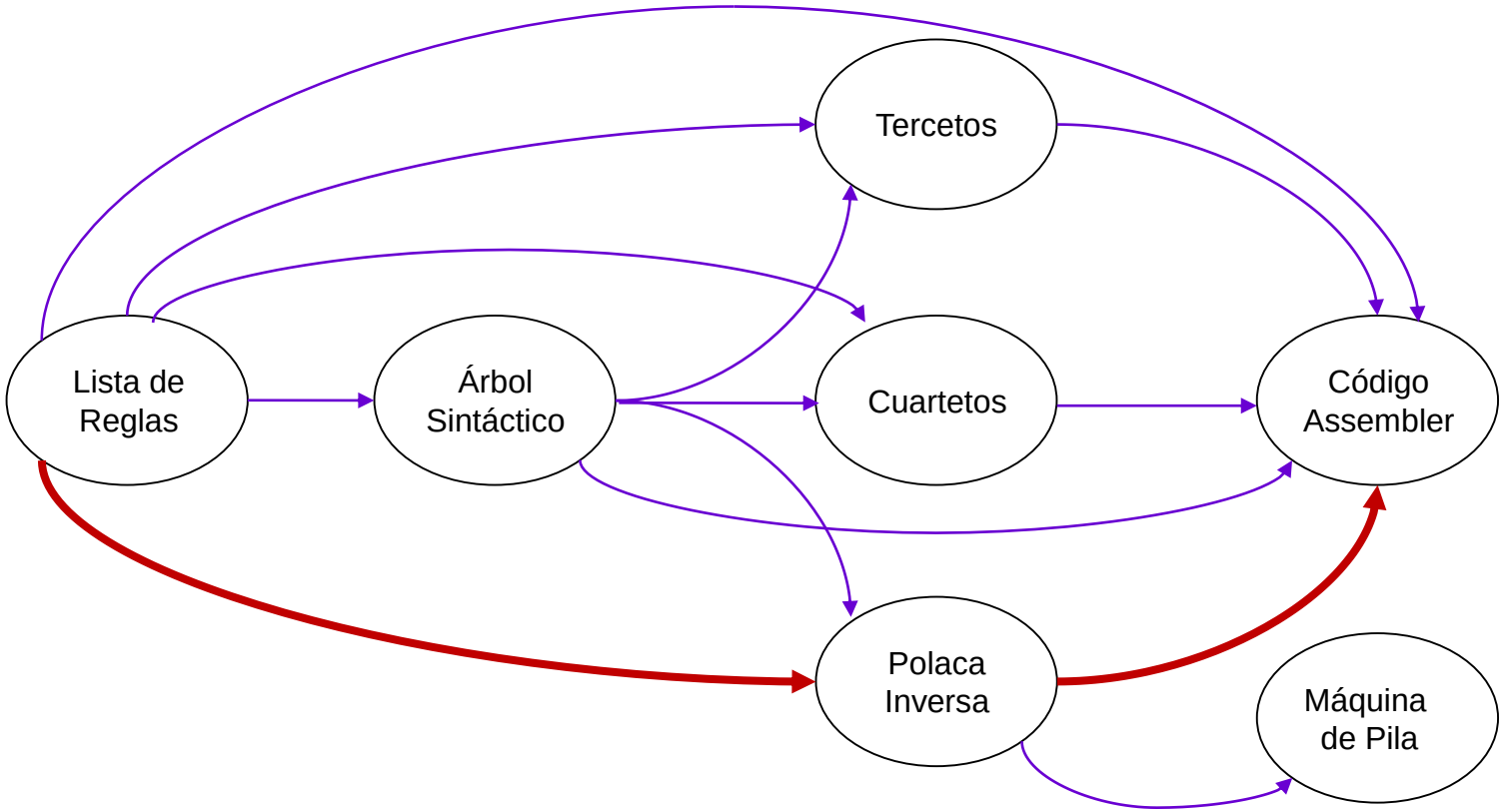


Salida ejecutable relativamente buena



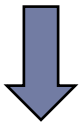
Lista de Reglas → Polaca Inversa → Assembler

Generación de Código



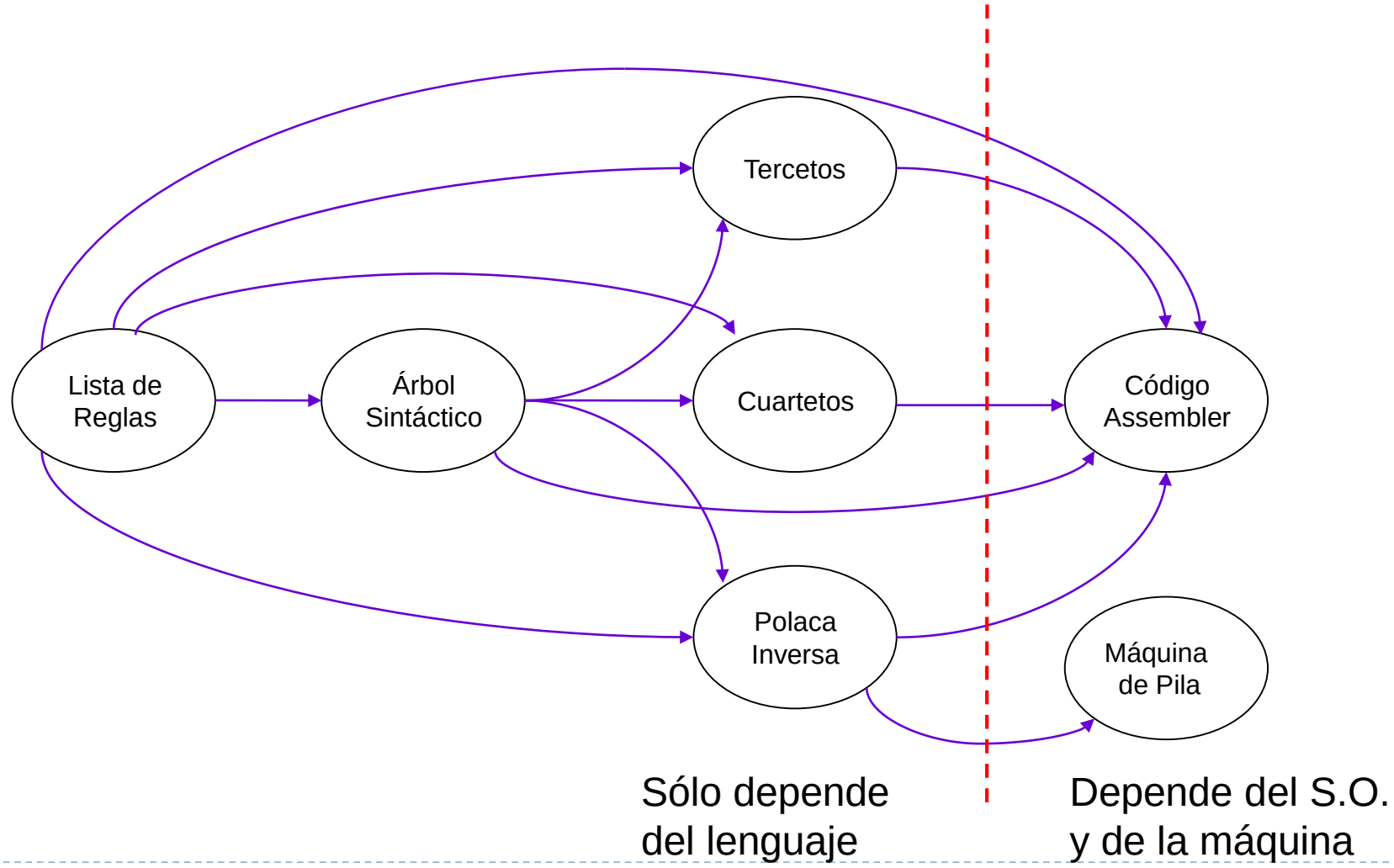
Comparación de caminos

Portabilidad



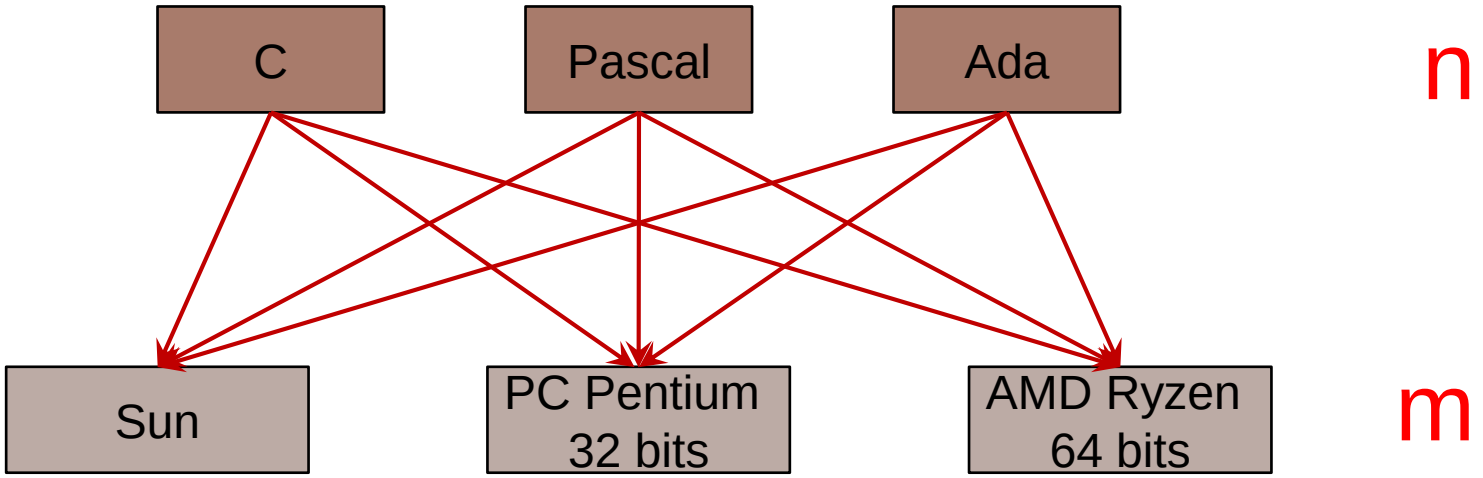
Representaciones Intermedias

Portabilidad



Portabilidad

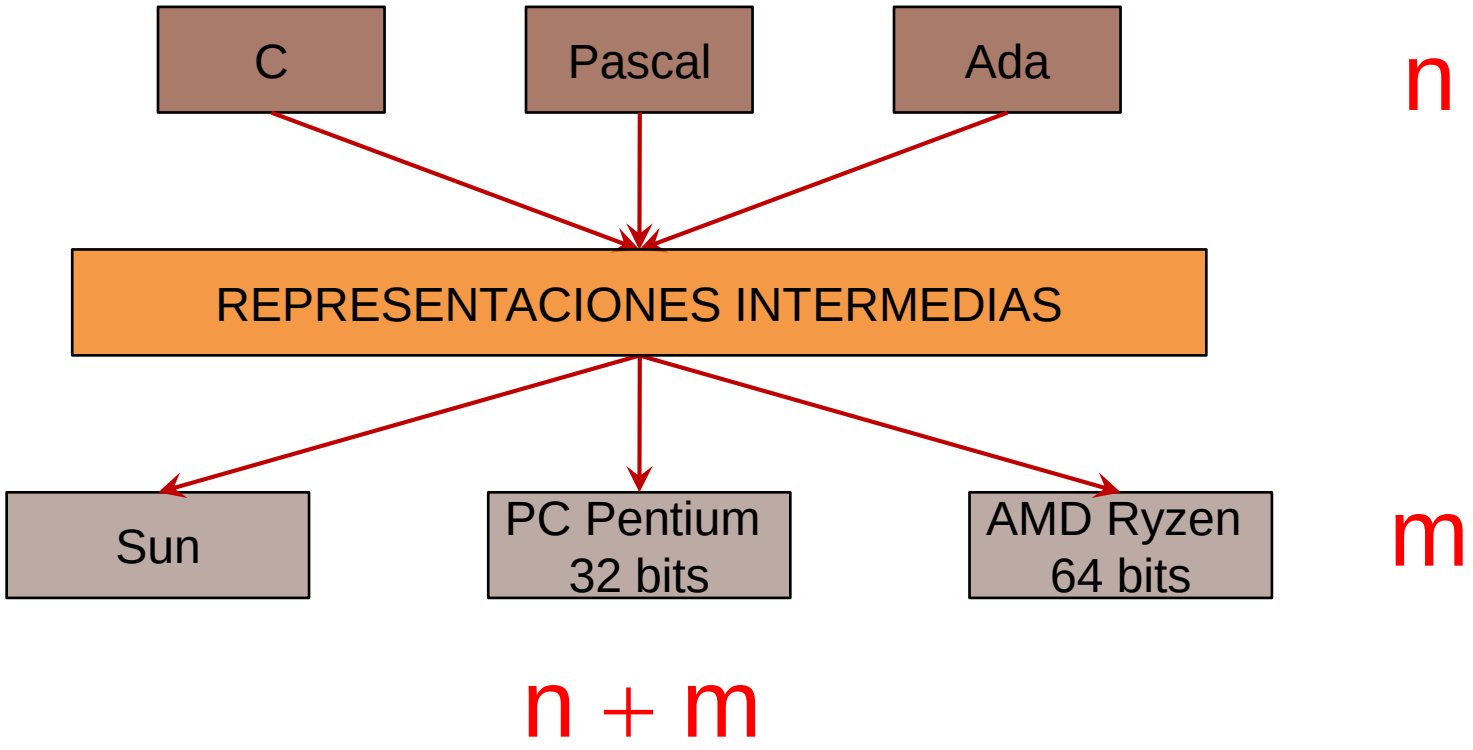
Lista de Reglas → Assembler



$n * m$

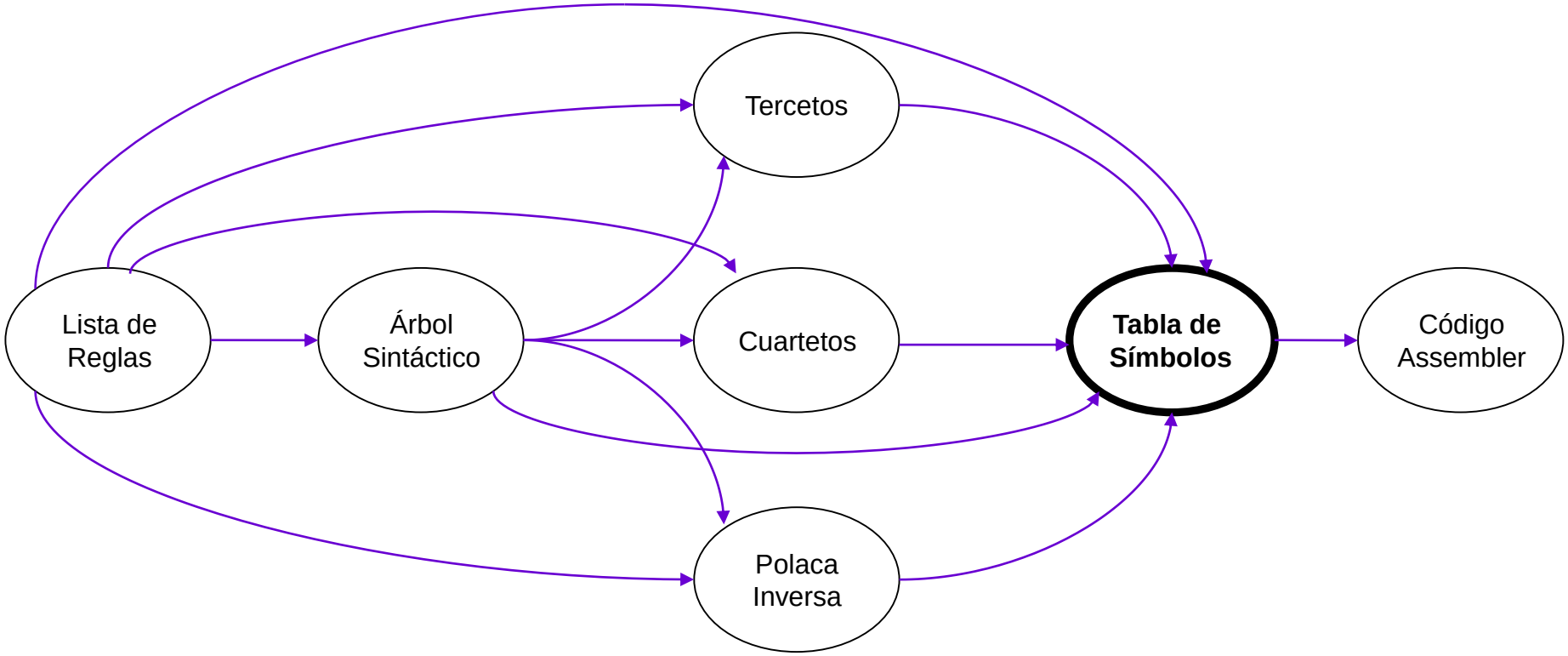
Portabilidad

L.Reglas → Representaciones Intermedias → Assembler



Generación de Código
Grafo 2: Sentencias declarativas

Sentencias declarativas
int x, y;



Comparación de caminos
Grafo 2: Sentencias declarativas

Mejor camino



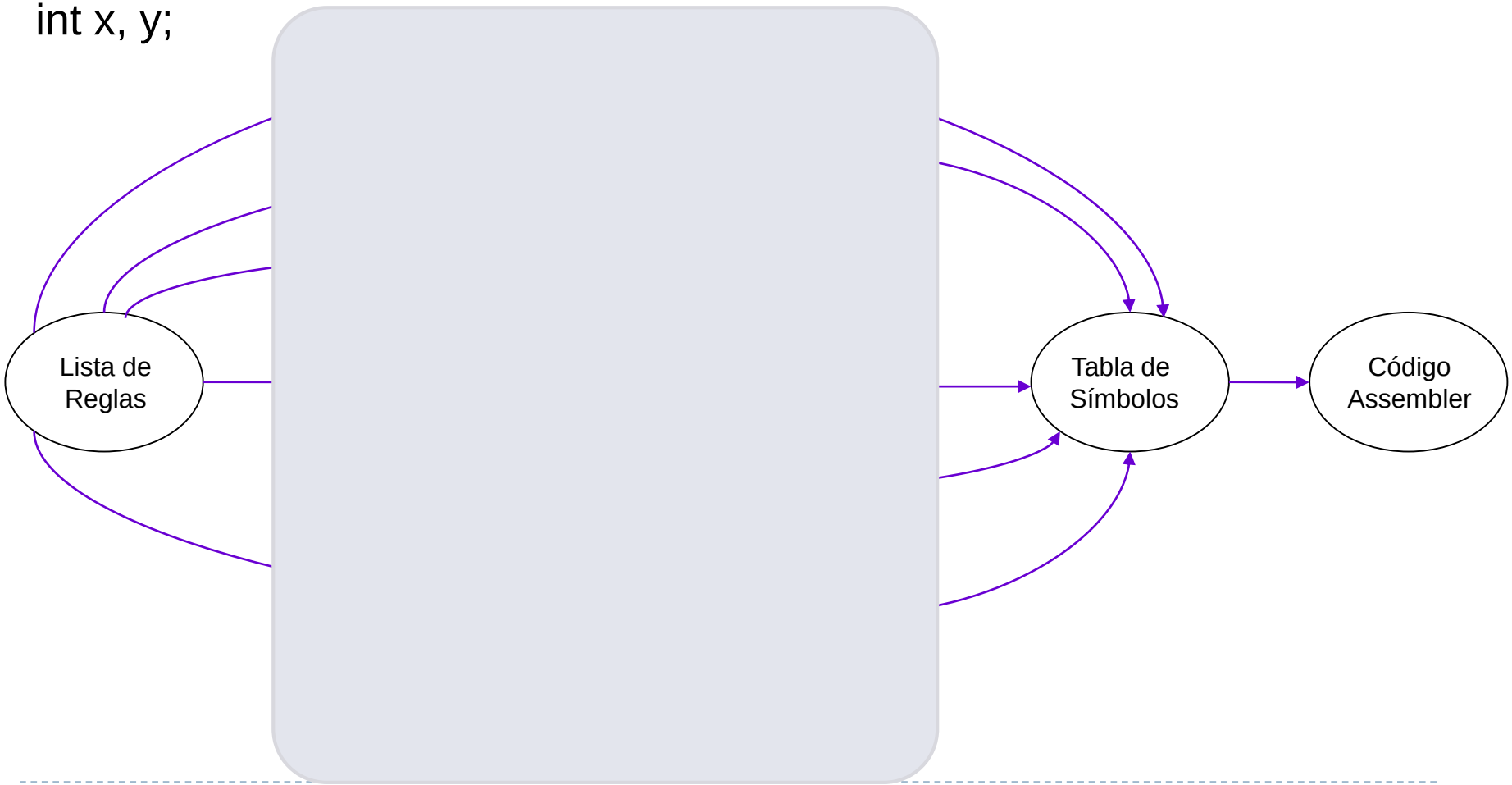
L. Reglas → Tabla de Símbolos → Assembler

(todo depende del lenguaje)

- Utilizar representaciones intermedias para sentencias declarativas:
  - No mejora la portabilidad
  - Compilador más lento

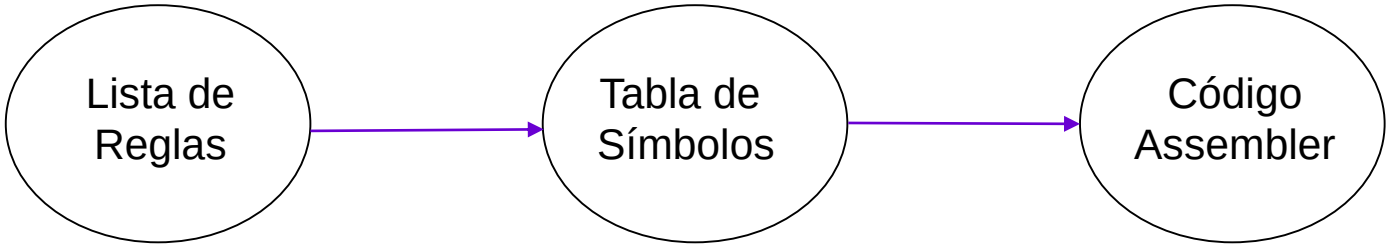
Generación de Código
Grafo 2: Sentencias declarativas

Sentencias declarativas
int x, y;



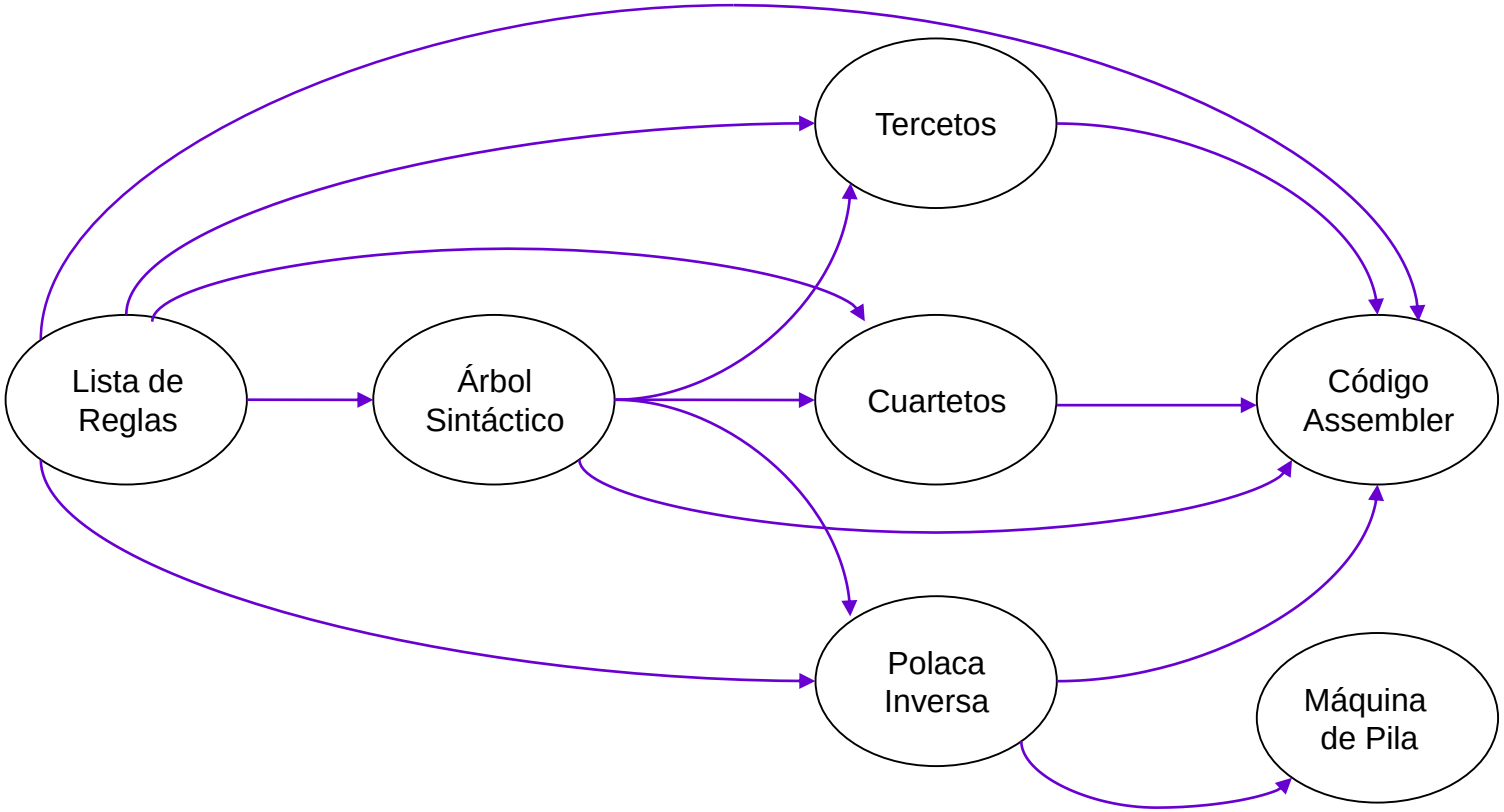
Generación de Código
Grafo 2: Sentencias declarativas

Sentencias declarativas
int x, y;



Generación de Código
Grafo 1: Sentencias Ejecutables

Sentencias ejecutables
x := x + 2;



¿Preguntas?

